Machine Learning in Factor Investing

DDA3600: Factor Investing

Chuan Shi
CUHK(SZ), Liangxin

Fall 2025

Table of Contents

- Consider the essential problem in factor investing / empirical asset pricing:

$$R^e_{i,t+1} = \mathbb{E}[R^e_{i,t+1}|\boldsymbol{x}_{i,t}] + e_{i,t+1} \quad \text{with} \quad \mathbb{E}[R^e_{i,t+1}|\boldsymbol{x}_{i,t}] = f(\boldsymbol{x}_{i,t}).$$

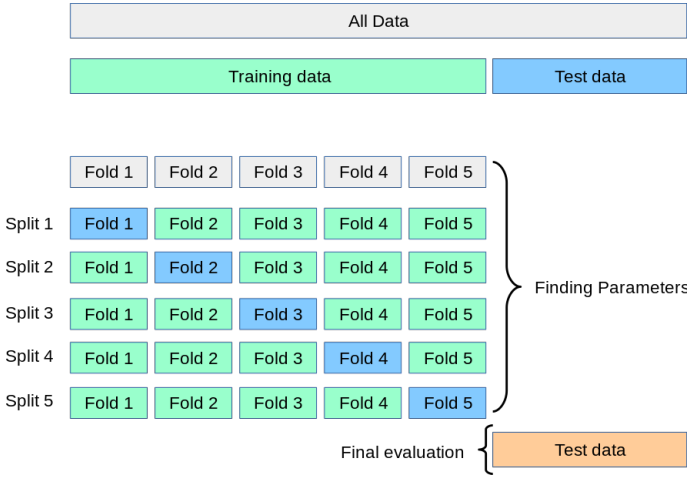
- Select a class of models (e.g., neural networks) $\hat{f} \in \mathcal{F}$, and learn its parameters from the data under a given loss function L. To guard against overfitting, an important concept is **regularization**:

$$\min \frac{1}{n} \sum_{i=1}^{n} L(\hat{f}(\boldsymbol{x}_i), y_i) + S(\hat{f}).$$

  where $S(\hat{f})$ is the regularization term.

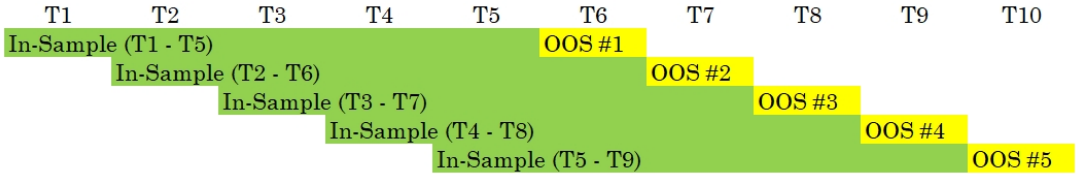
- Regularization procedures rely on the choice of hyperparameters. These hyperparameters are tuned via validation.

- Steps:
  - Split data into k smaller subsets called *folds*.
  - Train the model on $k-1$ folds and validate it on the remaining fold.
  - Repeat this process k times, with each fold serving as the validation set once.
  - Compute the average performance across all k test folds.
- **The sequential nature of data is not explicitly accounted for.**

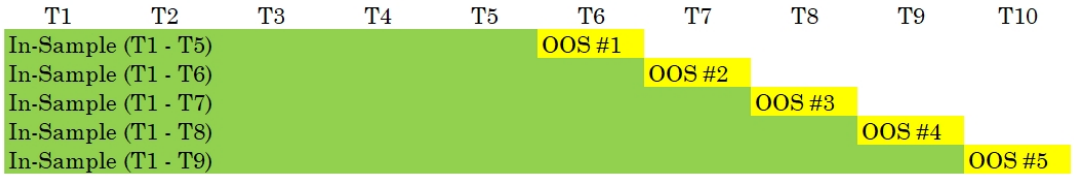


Source: https://scikit-learn.org/stable/modules/cross_validation.html

- Rolling WF:



- Expanding or Anchored WF:



Combinatorial Purged Cross-Validation (CPCV)

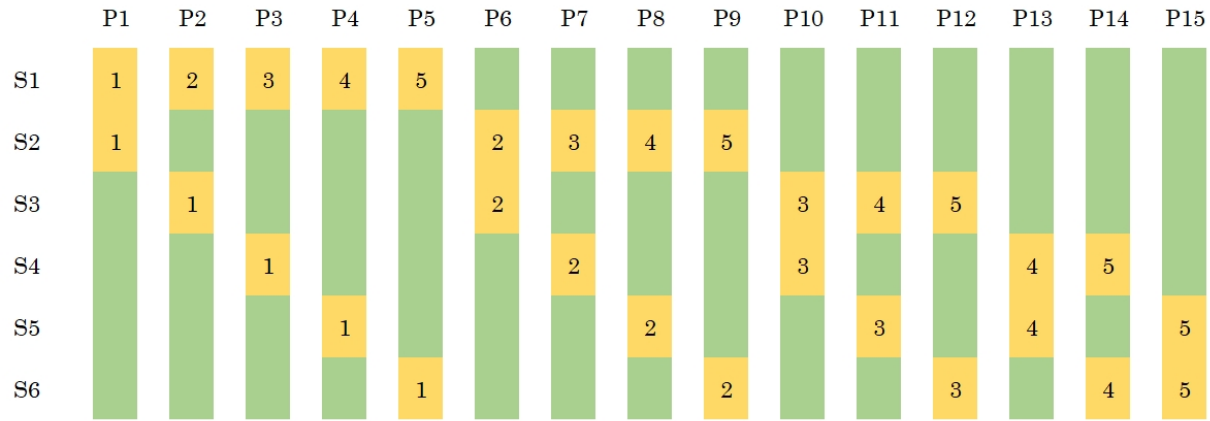
- Motivation: In WF and CV, each data point is only used in a **single test set**.
- CPCV (Lopez de Prado 2018) generates **multiple backtesting paths**, enabling robust performance evaluation across various market scenarios.
- Suppose data is split into N subsets and K of them are used for testing. Then, number of different paths is:

$$\left(\begin{array}{c} N \\ K \end{array} \right).$$

- Combines combinatorial splits with **purging** to address data leakage and dependency issues.

Multiple Backtesting Paths in CPCV

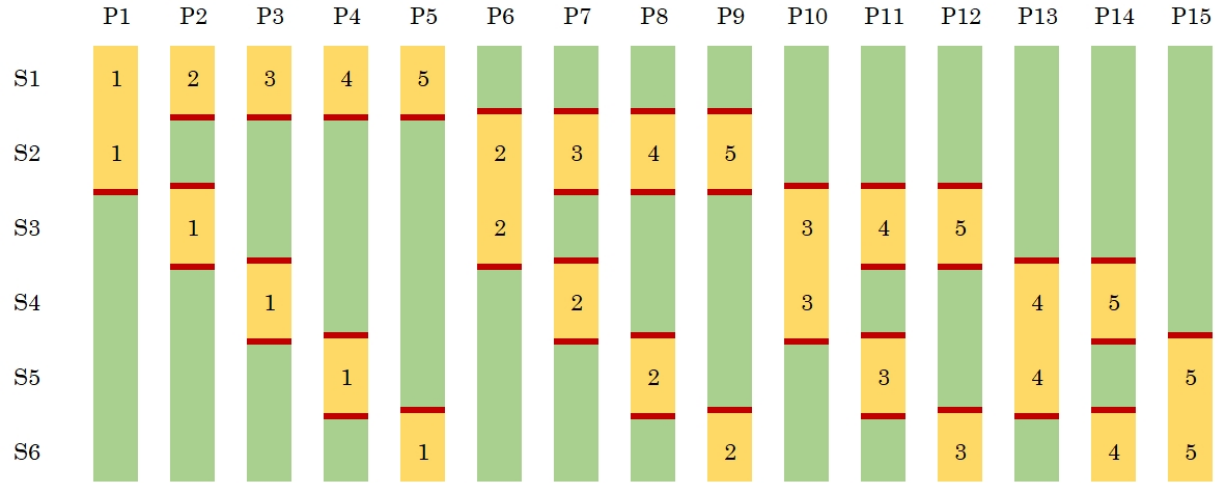
- Assume $N = 6$ and $K = 2$.



- **The Problem:** Sequential data points (e.g., x_{t-1}, x_t) are often highly correlated.
- **How It Happens:**
  - Training set includes x_{t-1}, test set includes x_t.
  - Model indirectly learns x_t from x_{t-1}, violating the unseen data principle.
- **Impact:** Overestimates performance; fails to reflect real-world scenarios.

Purging in CPCV

- Purging: Introduce a **buffer** between training and test sets.



- Steps:
  1. Divide the dataset into K subsets (folds).
  2. Select combinations of folds for training and testing.
  3. Purge adjacent periods between training and testing on the training side.
  4. Use all valid paths to train and evaluate the model.

- Outcome:
  - Comprehensive evaluation across multiple paths.
  - Robust estimates of out-of-sample performance.

CPCV vs. WF and CV

- Advantages:
  - Generates multiple paths for robust evaluation.
  - Mitigates data leakage with purging.
  - Provides a more reliable measure of strategy performance.

- Comparison with Traditional Methods:
  - **k-Fold Cross-Validation**:
    - Assumes i.i.d. data.
    - Fails to account for sequential dependencies.
  - **Walk-Forward Validation**:
    - Accounts for time order but produces only one backtesting path.
  - **CPCV**:
    - Combines the best of both worlds.
    - Creates multiple paths for robust validation.

- R-squared (similar to MSE):

$$\text{R-squared} = 1 - \frac{\sum_{(i,t)\in\mathcal{T}_{\text{val}}}(r_{i,t+1} - \hat{r}_{i,t+1})^2}{\sum_{(i,t)\in\mathcal{T}_{\text{val}}} r_{i,t+1}^2},$$

  where $\mathcal{T}_{\text{val}}$ indicates validation sample.

- Information Correlation (IC):

$$\text{IC} = \text{corr}(r_{i,t+1}, \hat{r}_{i,t+1}).$$

- Sharpe Ratio:
  - Form a portfolio with weights $w_{i,t+1} = \hat{r}_{i,t+1} / \sum_j |\hat{r}_{j,t+1}|$.
  - Evaluate the Sharpe ratio of this portfolio on the validation set.

Hyperparameter Tuning Criteria – The Choice Matters!

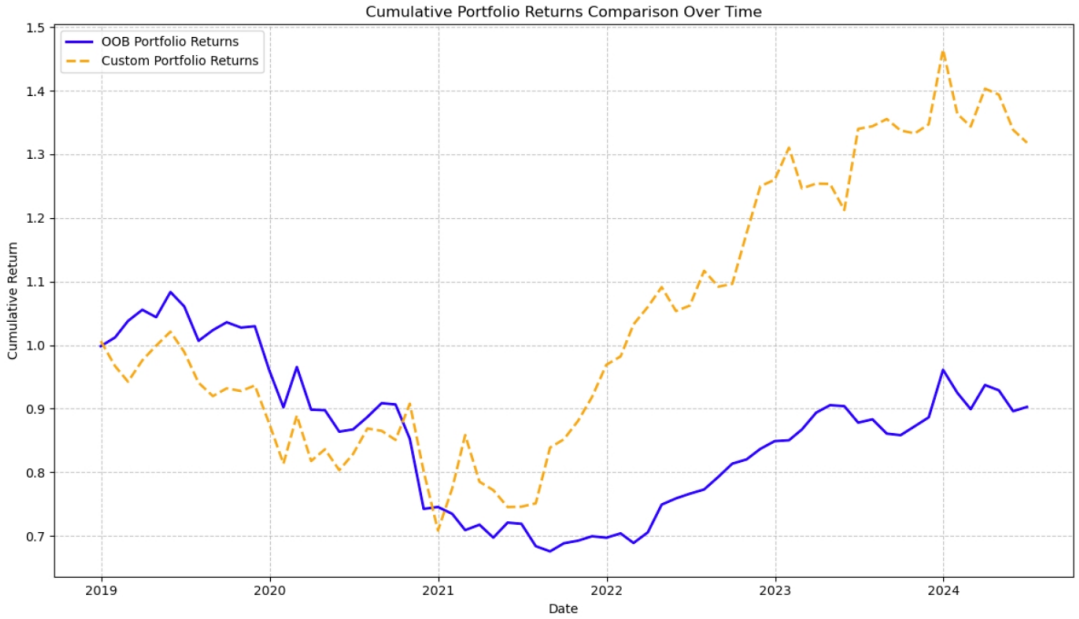


Table of Contents

2 Facts and Fictions

Fiction 1: Data will "Speak for Itself"

- Fallacy of Data Speaking for Itself:
  - Feeding raw financial data into ML algorithms without theoretical guidance often fails.
  - Financial data has low signal-to-noise ratio and non-stationarity, making out-of-the-box ML ineffective.
- Critical Considerations in Asset Pricing:
  - Prior distributions, scaling of covariates, regularization penalties, and tuning criteria significantly impact outcomes.
  - Example: Recall Nagel's example!
- Exhaustive permutation testing is infeasible (and I don't recommend it!); leverage financial theory for priors.

Fact 1: Apply Machine Learning within Asset Pricing Framework

- Integration of Machine Learning and Asset Pricing:
  - Leverage big data and machine learning within the framework of asset pricing theory.
  - Evolution from CAPM to APT/ICAPM and factor models has driven empirical progress.

- Common Themes:
  - Use of extensive covariates beyond traditional research.
  - Focus on out-of-sample (OOS) portfolio performance.
  - Modeling factor exposures (β) as functions of covariates (firm characteristics, macroeconomic variables).

- Unified Framework:
  - Despite different machine learning algorithms, all approaches align with SDF and multifactor models.
  - Demonstrates the synergy between advanced machine learning techniques and asset pricing theories.

Fiction 2: Machine Learning Models Neglect Interpretability

- Misconception: Machine learning models are often perceived as black boxes, leading to the belief that academic research neglects interpretability.

- Interpretability in Linear Models:
  - PCA (Kozak et al. 2018, 2020): First two principal components correspond to size and value factors.
  - IPCA (Kelly et al. 2019): Linear combination of managed portfolios through cross-sectional regression.

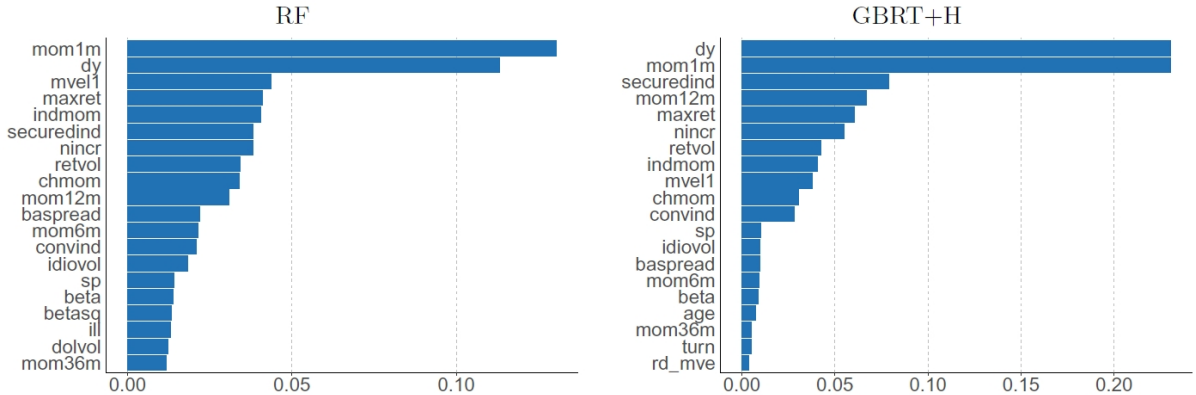
- Interpretability in Nonlinear Models:
  - While less intuitive, academic research emphasizes on feature importance (by, for example, permutation importance, gradients in a neural network, etc.).
  - See Fact 2!

Fact 2: Machine Learning Unveils Key Covariates

- Identifying Predictive Variables: - Machine learning helps uncover the most important predictive variables, aligning with empirical asset pricing findings.

- Examples:
  - Gu et al. (2020) use permutation Importance to Identify crucial covariates for expected returns, including momentum/reversal, liquidity-related, risk-related (e.g., idiosyncratic volatility), and fundamentals.
  - Chen et al. (2024) calculate partial derivatives of SDF weights to assess model interpretability.
  - Avramov et al. (2023) observe selected stocks across covariates to infer variable significance.

Fact 2: Machine Learning Unveils Key Covariates



Source: Gu et al. (2020)

Fiction 3: Complex Models Lead to Poor OOS Performance

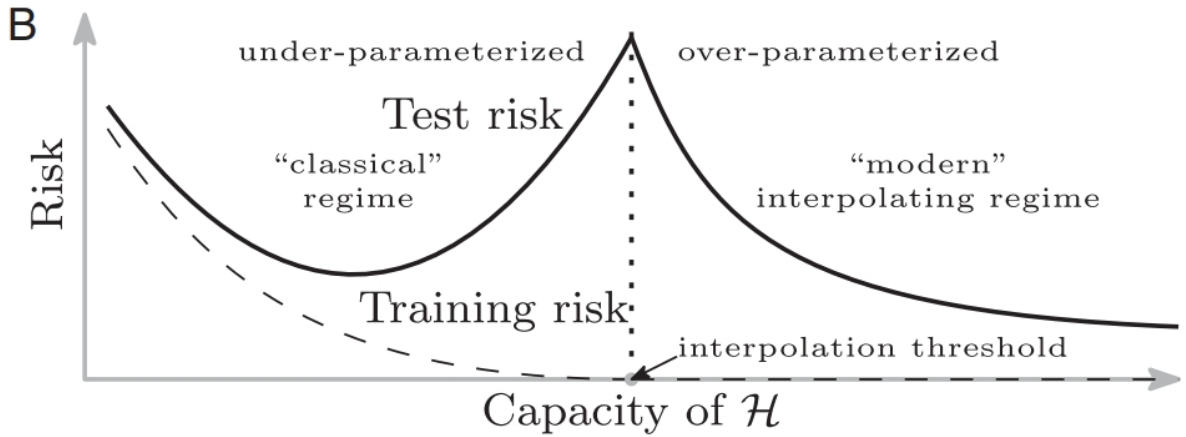
- Traditional View:
  - Complex models may overfit in-sample data, leading to poor OOS performance.
  - Bias-Variance Trade-off: Low complexity leads to underfitting (high bias, low variance); high complexity leads to overfitting (low bias, high variance).
- Latest Development:
  - "Double Descent" (Belkin et al. 2019; Hastie et al. 2022): Increasing parameters beyond training data size can improve OOS performance.
  - Empirical Evidence: Over-parameterized deep learning models generalize well despite high complexity.

Fact 3: With Regularization, Benefit > Cost

The true data-generating process (DGP) is unknown; all models are approximations of an unknown and potentially misspecified DGP.

Model	Simple			Complex
True	$K = 1000$	$K = 1000$	$\cdots$	$K = 1000$
Empirical	$K = 1$ (CAPM)	$K = 3$ (FF3)	$\cdots$	$K = 1000$

- Adding more variables comes with substantial statistical costs (over-parameterization).
- Compared to simple models, complex models provide a better approximation of the true DGP.
- Empirical results: better approximation of the DGP outweighs the statistical cost of adding parameters (Didisheim et al. 2023; Kelly et al. 2023).



Source: Belkin et al. (2019)

Fiction 4: Nonlinear Models Can Double SR Easily

- Misconception: Nonlinear models are often believed to easily double the Sharpe ratio compared to traditional linear factor models.

- Empirical Results (Baba-Yara et al. 2021):
  - Machine learning models show higher Sharpe ratios, but traditional models use fewer covariates, making comparisons unfair.
  - PCA related models (e.g. IPCA) achieves higher OOS Sharpe ratios than some nonlinear models (random forests, neural networks).

Machine Learning Models	OOS Sharpe Ratio
BPZ Random Forest	2.19
Davis Neural Network	3.21
KNS PCA	3.21
KPS IPCA	3.39

Fact 4: Nonlinear Models Offer Marginal Gains

- Nonlinear Relationships: Nonlinear relationships exist between covariates and asset returns.
- Interaction Terms:
  - Crucial in nonlinear models but impractical in traditional linear regression with many variables.
  - E.g., GAN of Chen et al. (2024) captures interactions but individual covariates' impact on SDF is nearly linear.
- Robustness and Tuning:
  - Fluctuations in Results: Significant across different empirical periods.
  - Research Protocol Needed: Lack of discussion on tuning and robustness under different hyperparameters.

Machine Learning Models	OOS Sharpe Ratio
BPZ Random Forest	0.86 (2.19)
Davis Neural Network	2.39 (3.21)
KNS PCA	2.77 (3.21)
KPS IPCA	3.21 (3.39)

- Bryan Kelly on machine learning:

> **Linear models are first order approximations. First order approximations are the most important approximations. When you try to set people's expectations of what you're going to get out of macine learning, you shouldn't expect rvolutionary performance from machine learning. You should expect second and third order of improvements, because the first order models we are use are good. They leave some predictability on the table because there are those non-linearities that we can exploit. But you should not expect it to double or triple your Sharpe ratio. Seeing something like a 20 percent increase in Sharpe Ratio from introducing non-linearities feels realistic to me.**

Fiction 5: ML Models Can Be Easily Applied in Practice

- Misconception: Machine learning models are often perceived as easily applicable in practice, but practical challenges exist.
- High Turnover Ratios:
  - Empirical results from Avramov et al. (2023):

Machine Learning Models	Monthly Turnover Ratio
Gu et al. (2020)	0.976
Chen et al. (2024)	1.664
Kelly et al. (2019)	1.186
Gu et al. (2021)	1.565

  - Traditional factors (e.g., size, value) have turnover rates below 10%. High turnover rates make it challenging to achieve excess net returns after costs.
- Jensen et al. (2022) integrate transaction costs into portfolio optimization.

Fact 5: Limited Value for Some Institutional Investors

- Machine learning models uncover predictability in stocks with high arbitrage and transaction costs, limiting their value for some institutional investors.

- Empirical Findings (Avramov et al. 2023):
  - Excess returns often come from short legs or tiny market cap stocks.
  - Anomalies are concentrated in small-cap stocks.
  - Significant performance decline in nonlinear models in subsamples (excluding tiny caps, unrated firms, distressed firms).
  - Linear models (e.g. IPCA) produce consistent performance across full sample and subsamples.

- Successfully adapting nonlinear models to focus on stocks with lower arbitrage and transaction costs is crucial for practical implementation.

Table of Contents

3 Penalized Regression

- Penalized regression modifies the OLS objective function by adding a penalty term:

$$\min_{\beta_1,\cdots,\beta_K} \sum_{i=1}^{N}(y_i - \beta_0 - \beta_1 x_{1i} - \cdots - \beta_K x_{Ki})^2 + \lambda \sum_{j=1}^{K} f(\beta_j),$$

  where λ is the regularization parameter. **Note that the intercept β_0 has been left out of the penalty term.**

- OLS does not impose any restriction on the number or size of the regression coefficients, and this can lead to overfitting; Penalized regression can be used to address the problem of overfitting.

- From a Bayesian perspective, this brings *prior knowledge* to the estimation process.

- The penalty term pulls the value of a coefficient closer to zero, reducing its impact on the model. This process is called *shrinkage*. In extreme cases, the penalty term can shrink a coefficient to zero, effectively removing the corresponding covariate from the model (i.e., it achieves *variable selection*).

- Ridge regression (Hoerl and Kennard 1970) adds a penalty proportional to the sum of the squares of the coefficients (ℓ_2 regularization), i.e., $f(\beta_j) = \beta_j^2$, and the objective function in matrix form becomes (assuming no intercept):

$$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda(\boldsymbol{\beta}^\top \boldsymbol{\beta}).$$

- The estimator:

$$\hat{\boldsymbol{\beta}}^{\text{Ridge}} = \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_K\right)^{-1} \mathbf{X}^\top \boldsymbol{y}.$$

- The hyperparameter λ constrols for the amount of regularization.

- Including λ makes $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_K)$ nonsingular even when $\mathbf{X}^\top \mathbf{X}$ is noninvertible.

$$\hat{\boldsymbol{\beta}}^{\text{Ridge}} = \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_K\right)^{-1} \mathbf{X}^\top \boldsymbol{y}.$$

- As $\lambda \to 0$, it reduces to OLS estimator; while as $\lambda \to \infty$, $\hat{\boldsymbol{\beta}}^{\text{Ridge}} \to 0$.
- Ridge regression shrinks the coefficients but does not set any of them exactly to zero, retaining all covariates. (**No sparse representation.**)
- Validation is employed to tune λ. The validation set chooses λ that provides the smallest loss function (e.g., MSE).
- The Ridge estimator is **biased**, i.e., $\mathbb{E}[\hat{\boldsymbol{\beta}}^{\text{Ridge}}] \neq \boldsymbol{\beta}$. But does it matter? No, if it produces better performance OOS.

- Suppose the prior on $\boldsymbol{\beta}$ follows multivariate normal:

$$\boldsymbol{\beta} \sim \mathcal{N}\left(\mathbf{0}, \sigma_0^2 \mathbf{I}_K\right).$$

- With normal prior and likelihood, the posterior mean of $\boldsymbol{\beta}$ is:

$$\hat{\boldsymbol{\beta}}^{\text{posterior}} = \left(\mathbf{X}^\top \mathbf{X} + \frac{\sigma^2}{\sigma_0^2} \mathbf{I}_K\right)^{-1} \mathbf{X}^\top \boldsymbol{y}.$$

- The hyperparameter λ in Ridge plays the role of σ^2/σ_0^2 in this Bayesian framework, which combines prior views with the likelihood function.

- There is **no bias** from a Bayesian perspective because $\mathbb{E}\left[\hat{\boldsymbol{\beta}}^{\text{Ridge}}\right]$ is the posterior mean of $\boldsymbol{\beta}$.

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

- Lasso (Least Absolute Shrinkage and Selection Operator) regression (Tibshirani 1996) adds a penalty proportional to the sum of the absolute values of the coefficients (ℓ_1 Regularization), i.e., $f(\beta_j) = |\beta_j|$, and the objective function in matrix form becomes (assuming no intercept):

$$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \|\boldsymbol{\beta}\|_1.$$

- There is no analytical solution for Lasso estimator.

- Lasso regression can shrink some coefficients to exactly zero, effectively performing variable selection and producing more interpretable models.

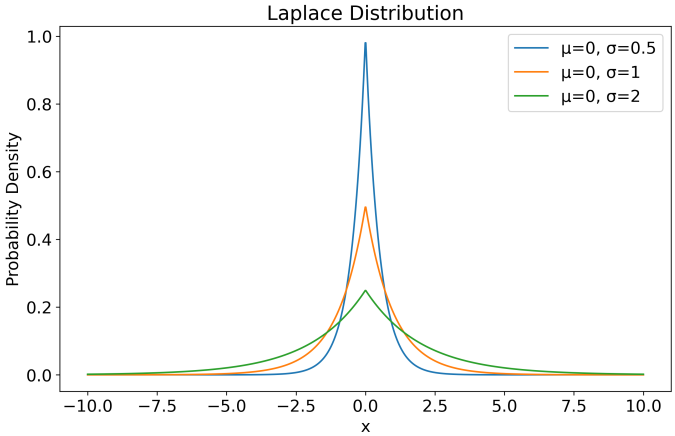
- Both Ridge and Lasso have solutions even when $\mathbf{X}^\top \mathbf{X}$ may not be of full rank or ill conditioned.

- Lasso can be interpreted as imposing a **Laplace prior** on the coefficients $\boldsymbol{\beta}$:

$$P(\boldsymbol{\beta}|\lambda) \propto \left(\frac{\lambda}{2\sigma}\right)^K \exp\left(-\frac{\lambda\|\boldsymbol{\beta}\|_1}{\sigma}\right),$$

  where:
  - λ: Controls the dispersion of the prior.
  - σ: Noise standard deviation.

- Laplace distribution has a spike at zero and concentrates probability mass closer to zero.



- We can view Lasso as a **maximum a posteriori** (MAP) estimator:

$$\hat{\boldsymbol{\beta}}^{\text{Lasso}} = \arg\max_{\boldsymbol{\beta}} p(\boldsymbol{\beta}|\boldsymbol{y}) = \arg\max_{\boldsymbol{\beta}} p(\boldsymbol{y}|\boldsymbol{\beta})p(\boldsymbol{\beta}).$$

- Given the shape of Laplace prior, this leads to sparse solutions, where some coefficients β_j are exactly zero.

- As a comparison, Ridge uses ℓ_2 regularization (Gaussian prior), which does not produce sparsity.

- In contrast, the maximum likelihood estimator would just maximize $p(\boldsymbol{y}|\boldsymbol{\beta})$, without weighting by the prior distribution. In the case of a flat prior, the MAP and maximum likelihood estimator coincide.

- The elastic net is another regularization and variable selection method, where it combines ℓ_1 and ℓ_2 norm penalties:
$$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda_1 \|\boldsymbol{\beta}\|_1 + \lambda_2 \|\boldsymbol{\beta}\|_2^2.$$
- It still generates sparse representations, making it possible to use model selection criteria for choosing the optimal hyperparameters. (But you can use validation as well!)
- It promotes a grouping effect, where covariates with strong correlations are either included in or excluded from the model collectively.

- Lasso forces coefficients to be equally penalized. But we can assign different weights to different coefficients, which leads to the adaptive Lasso:

$$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \sum_{j=1}^{K} w_j |\beta_j|.$$

- Weights w_j can be chosen as $w_j = 1/|\beta_j|^\gamma$, where β_j is the OLS estimate. Why?

This choice of weights ensures that covariates with larger OLS coefficients receive less penalty, allowing them to retain their influence in the model, while covariates with smaller OLS coefficients are more heavily penalized, encouraging sparsity.

- The adaptive Lasso is a convex optimization problem and thus does not suffer from multiple local minima.

- In quantitative investment, you can divide covariates (i.e., firm characteristics) into accounting versus market or technical versus fundamental.
- Group Lasso: Divide K covariates into L groups. Model coefficients are $\boldsymbol{\beta} = [\boldsymbol{\beta}_1^\top, \boldsymbol{\beta}_2^\top, \ldots, \boldsymbol{\beta}_L^\top]^\top$, where $\boldsymbol{\beta}_l$ is the coefficient vector for the l-th group.
- Group Lasso solves the following problem:

$$\mathcal{L}(\boldsymbol{\beta}) = \left(\boldsymbol{y} - \sum_{l=1}^{L} \mathbf{X}_l \boldsymbol{\beta}_l\right)^\top \left(\boldsymbol{y} - \sum_{l=1}^{L} \mathbf{X}_l \boldsymbol{\beta}_l\right) + \lambda \left(\sum_{l=1}^{L} \boldsymbol{\beta}_l^\top \boldsymbol{\beta}_l\right)^{1/2}.$$

- Group-Level Selection:
  - If $\|\boldsymbol{\beta}_l\|_2 > 0$, the l-th group is selected, and all variables in the group are retained.
  - If $\|\boldsymbol{\beta}_l\|_2 = 0$, the l-th group is excluded, and all variables in the group are removed.

The Group Lasso: Within-Group Sparsity

- Group Lasso does not guarantee within-group sparsity.
- If a group is selected, all variables in the group are retained, even if some coefficients are close to zero.
- **Sparse** Group Lasso: Introduce sparsity at both group and variable levels:

$$\mathcal{L}(\boldsymbol{\beta}) = \left(\boldsymbol{y} - \sum_{l=1}^{L} \mathbf{X}_l \boldsymbol{\beta}_l\right)^\top \left(\boldsymbol{y} - \sum_{l=1}^{L} \mathbf{X}_l \boldsymbol{\beta}_l\right) + \lambda_1 \left(\sum_{l=1}^{L} \boldsymbol{\beta}_l^\top \boldsymbol{\beta}_l\right)^{1/2} + \lambda_2 \sum_{j=1}^{K} |\beta_j|.$$

- It achieves both group-level and variable-level selections.

- Bridge regression (Frank and Friedman 1993) generalizes Lasso and Ridge to ℓ_q penalty:

$$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \sum_{j=1}^{K} |\beta_j|^q.$$

  It is immediate clear that $q = 0, 1$, and 2 correspond to OLS, Lasso and Ridge.

- From a machine learning perspective, q is a hyperparameter to be selected, and the solution is sparse for $0 \leq q \leq 1$.

- When the solution is sparse – use model selection criteria to pick hyperparameters (both λ and q). Otherwise, use a validation sample.

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

- Lasso, adaptive Lasso, group Lasso, Ridge, Bridge, and Elastic net are all linear or parametric approaches for shrinkage.

Model	Objective Function
Lasso	$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \|\boldsymbol{\beta}\|_1$
Adaptive Lasso	$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \sum_{j=1}^{p} w_j \lvert\beta_j\rvert$
Group Lasso	$\mathcal{L}(\boldsymbol{\beta}) = \left(\boldsymbol{y} - \sum_{l=1}^{L} \mathbf{X}_l \boldsymbol{\beta}_l\right)^\top \left(\boldsymbol{y} - \sum_{l=1}^{L} \mathbf{X}_l \boldsymbol{\beta}_l\right) + \lambda \left(\sum_{l=1}^{L} \boldsymbol{\beta}_l^\top \boldsymbol{\beta}_l\right)^{1/2}$
Ridge	$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \|\boldsymbol{\beta}\|_2^2$
Bridge	$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \sum_{j=1}^{K} \lvert\beta_j\rvert^q$
Elastic Net	$\mathcal{L}(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}) + \lambda_1 \|\boldsymbol{\beta}\|_1 + \lambda_2 \|\boldsymbol{\beta}\|_2^2$

OOS Performance: OLS vs. Elastic Net

- **Covariates**: 54 firm characteristics are used as covariates.
- **Empirical Period**: 2021/1 – 2024/7, monthly frequency.
- **Stock Universe**: Stocks in the CSI 500 Index.
- **Model Training**:
  - Use the past 36 months of data to estimate parameters or train models (OLS and Elastic Net).
  - For Elastic Net, use 5-fold cross-validation to tune hyperparameters.
- **Portfolio Construction**:
  - Predict next month's returns using the trained models.
  - Construct portfolios based on the predicted returns using portfolio sorting.
- **Rolling Window Training**: Repeat the process using a rolling 36-month window to construct out-of-sample portfolios.

OOS Performance: OLS vs. Elastic Net

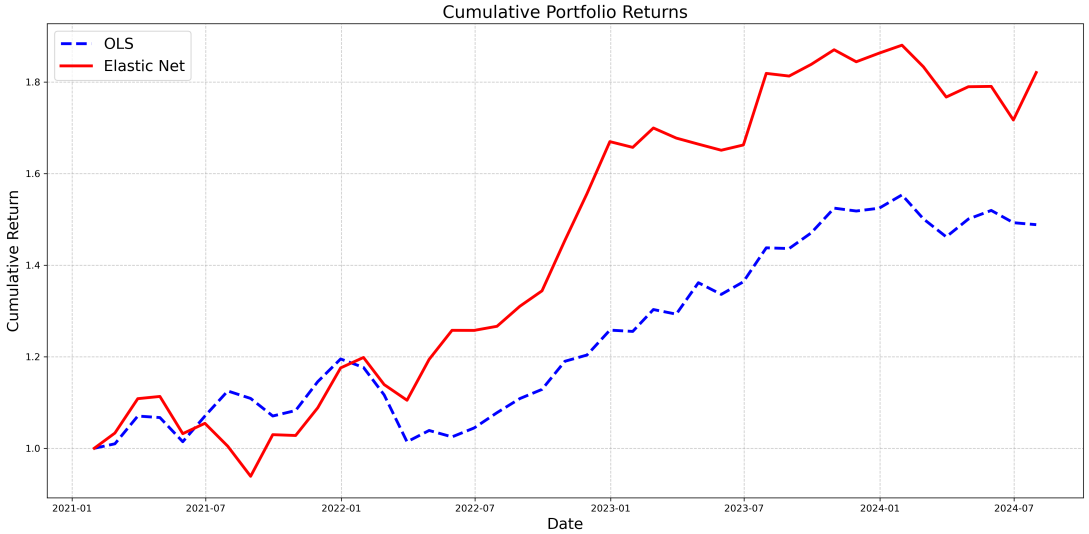


Table of Contents

- Ways to learn about f:

Assumption	Econometrics	Machine Learning
$f(\boldsymbol{x}^i) = \text{avg}(y^j)_{\boldsymbol{x}^j \text{ near } \boldsymbol{x}^i}$	portfolio sort	regression trees, random forest, boosting
$f(\boldsymbol{x}^i)$ linear	OLS/GLS, Fama-MacBeth regression, regression based prediction	Penalized regression, e.g., ridge, Lasso
$f(\boldsymbol{x}^i)$ nonlinear	nonlinear least squares, GMM, MLE	neural networks, deep learning

Table adapted from Lasse Heje Pedersen's Bid Data Asset Pricing Lecture 5: Machine Learning in Asset Pricing.

Introducing Nonlinearity

- Traditional asset pricing models (e.g., CAPM, Fama-French) and quantitative investment strategies based on factor models assume **linear relationships** between asset returns and covariates.
- However, financial markets often exhibit **nonlinear behaviors** due to:
  - Complex investor psychology
  - Market shocks and extreme events
  - Asymmetric and threshold effects
- Nonlinearity is crucial for accurate predicting asset returns and risk management in quantitative investment.

- **Polynomial Terms**
  - Higher-order powers of covariates (e.g., x^2, x^3) capture curvature in relationships.
  - Example: Barra has a nonlinear size factor (size^3) that captures the middle size effect on returns.

- **Interaction Terms**
  - Products of covariates (e.g., $x_1 \times x_2$) capture joint effects.
  - Example: See next few slides.

Empirical evidence suggests that interactions between covariates are the most important source of nonlinearity in predicting future returns.

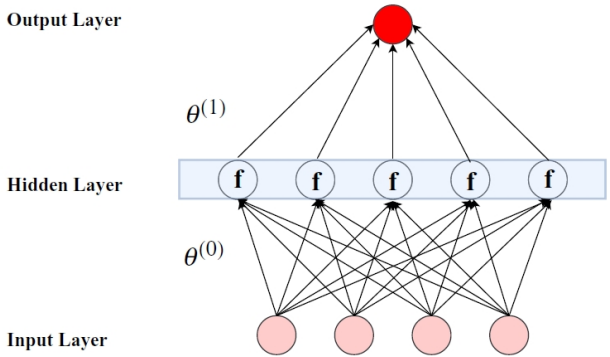
Why Interaction Matters

- **Market Behavior**
  - Investor sentiment and herding effects are inherently nonlinear.
  - Market reactions to shocks are often asymmetric.
- **Asset Returns**
  - Volatility clustering and leverage effects.
  - Tail risks and extreme events (e.g., crashes, bubbles).
- **Risk Factors**
  - Nonlinear relationships between asset prices and risk factors.
  - Interaction effects between multiple covariates.

Table of Contents

5 Neural Networks

- Neural networks (NNs), modeled after the structure of the human brain, are composed of layers of "neurons" interconnected by "synapses" that carry signals between the layers.
- The network handles information by passing it from an input layer, through one or more hidden layers, to an output layer, which is commonly referred to as a Feed-Forward Network (FFN).
- The output layer makes predictions similar to fitted values in regression analysis.



Source: Gu et al. (2020)

- Input layer: each neuron k is just the kth feature, i.e., $a_k^{(0)} = x_k$.
- Hidden layer $h = 1, \cdots, H$: each neuron x_j^h is a (non)linear *activation* function g of prior neurons:

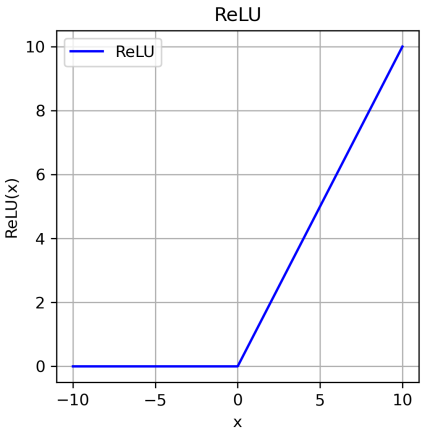
$$a_j^{(h)} = g\left(\theta_{j,0}^{(h-1)} + \sum_i \theta_{j,i}^{(h-1)} a_i^{(h-1)}\right).$$

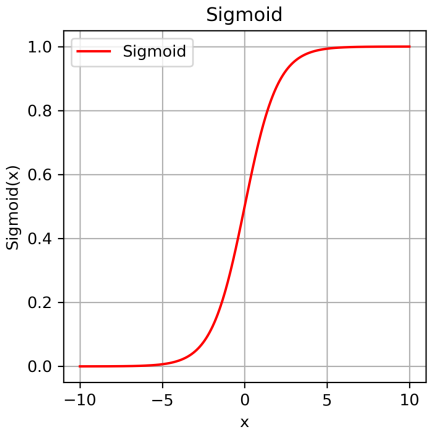
- Output layer (i.e., the prediction):

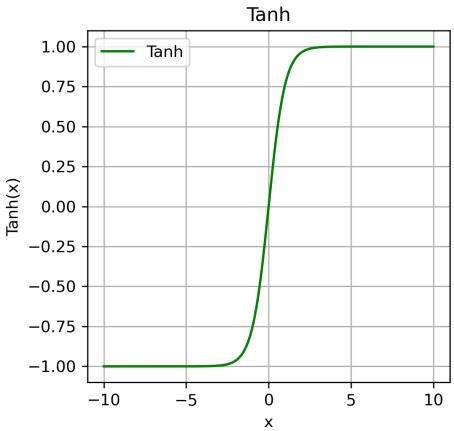
$$\hat{f}(x) = \theta_0^{(H)} + \sum_i \theta_i^{(H)} a_i^{(H)}.$$

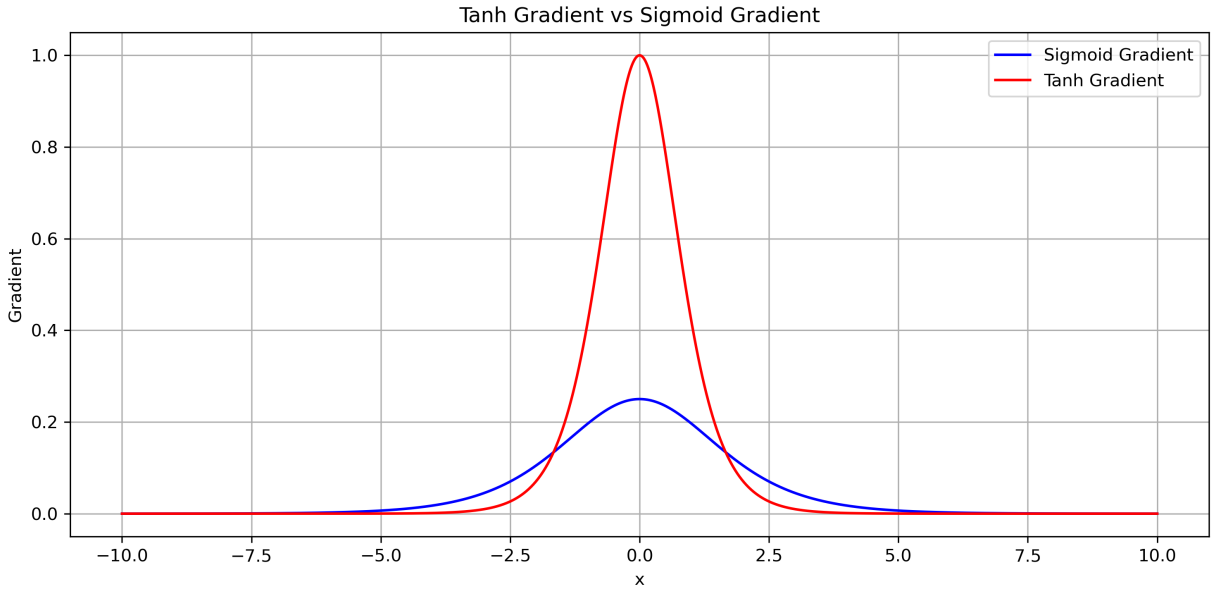
- At each neuron, we first compute a linear function of past neurons (with weights $\boldsymbol{\theta}$), and then transform it with the activation function g.

$\text{ReLU}(x) = \max(0, x)$ Sigmoid: $s(x) = \dfrac{1}{1+e^{-x}}$ tanh: $2s(2x) - 1$









- Taking ReLU as an example, the neural network can be written as:

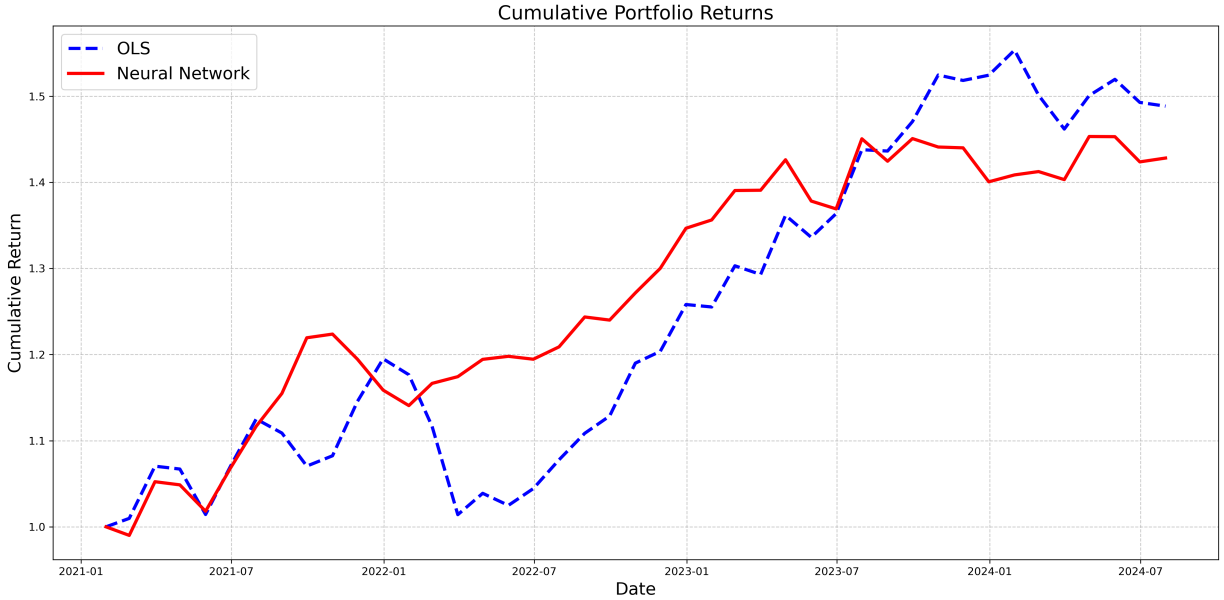
$$\text{output} = \max\left(\max\left(\max\left(\mathbf{A}^{(0)}\mathbf{\Theta}^{(0)}, 0\right)\mathbf{\Theta}^{(1)}, 0\right)\cdots\mathbf{\Theta}^{(H-1)}, 0\right)\mathbf{\Theta}^{(H)}.$$

  Yes, there are a lot of max!!!

- We run an optimization to minimize the loss function. For continuous predicted variable, mean squared errors (MSE) is the default choice. (But is this always a good choice for finance?)

- If the activation function is linear – simply ignore the MAX operator in the above equation. **NN boils down to OLS.**

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.



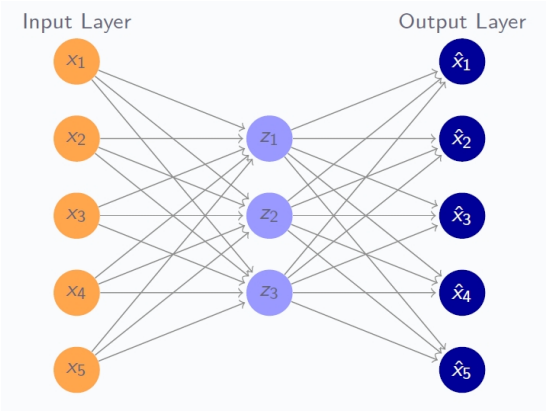
- Autoencoders are a type of *unsupervised* learning model that perform dimensionality reduction by compressing input data into a lower-dimensional representation, then reconstructing the original input from this compressed version.
- Useful for extracting latent features from complex data, e.g., asset pricing applications.
- Training autoencoders involves the minimization error from reconstructing its inputs $\boldsymbol{x}$ in two steps:

$$\boldsymbol{x} \xrightarrow{f(\boldsymbol{x})} \boldsymbol{z} \xrightarrow{g(\boldsymbol{z})} \hat{\boldsymbol{x}}$$

  - Encoding $\boldsymbol{x} \xrightarrow{f(\boldsymbol{x})} \boldsymbol{z}$: Compress input data into a lower-dimensional representation.
  - Decoding $\boldsymbol{z} \xrightarrow{g(\boldsymbol{z})} \hat{\boldsymbol{x}}$: Reconstruct the original data from the compressed representation.

- **Input Layer (orange):** Represents the original input variables.
- **Hidden Layer (purple):** Encodes input data into a compressed representation.
- **Output Layer (blue):** Reconstructs the input data from the compressed representation.

- Orange → Purple: Encoding process.
- Purple → Blue: Decoding process.



- Given the choice of activation functions, f and g, and encoding dimension, K, train an autoencoder to minimize the reconstruction error:

$$\frac{1}{NT}\|\mathbf{X} - \hat{\mathbf{X}}\|_F^2$$

  where $\|\cdot\|_F$ is the Frobenius norm.

- Add regularization terms to penalize weights.

- Increase network depth by introducing additional hidden layers around the encoding layer.

- **Does minimizing $\|\mathbf{X} - \hat{\mathbf{X}}\|_F^2$ remind you of PCA?**

- Linear Autoencoder:

$$\min_{\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{N \times K}} \|\mathbf{X} - \mathbf{X}\mathbf{W}_1\mathbf{W}_2\|_F^2.$$

  - $\mathbf{W}_1$: Encoder matrix for dimensionality reduction.
  - $\mathbf{W}_2$: Decoder matrix for reconstruction.
  - No orthogonality or unit norm constraints on $\mathbf{W}_1$ and $\mathbf{W}_2$.

- PCA:

$$\min_{\mathbf{W} \in \mathbb{R}^{N \times K}} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2, \quad \text{s.t. } \mathbf{W}^\top\mathbf{W} = \mathbf{I}.$$

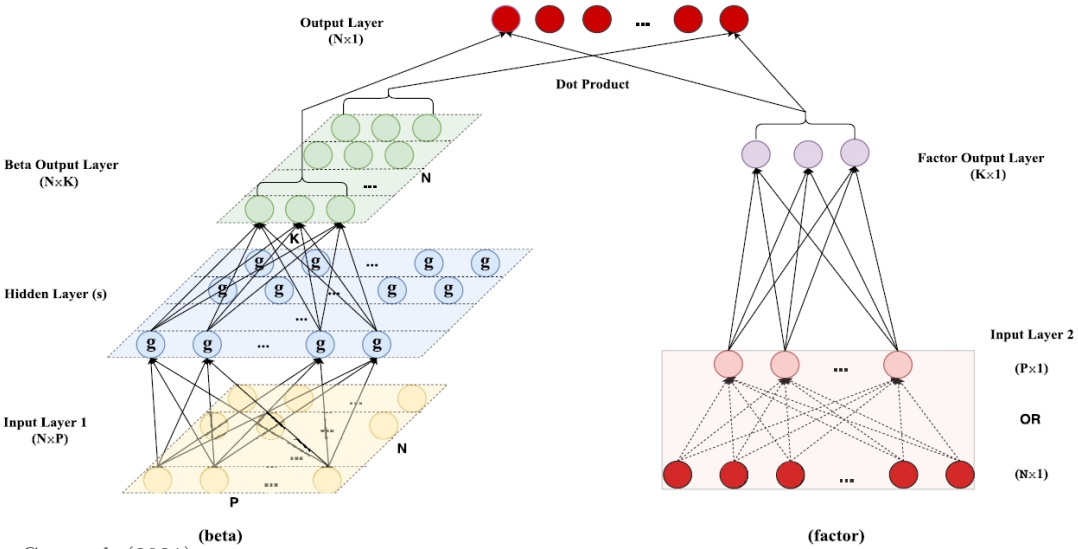
  - $\mathbf{W}$: Projects high-dimensional data to a lower-dimensional subspace.
  - $\mathbf{W}^\top$: Reconstructs the data back to the original space.
  - Orthogonality constraint ensures unit norm and minimum reconstruction error.

Conditional Autoencoding – Motivation

- **Traditional Factor Models:**
  - Assume *static* β – time-invariant factor exposures.
  - Limited ability to capture dynamic market conditions.

- **Real-World Financial Markets:**
  - Asset risk exposures (β) are time-varying.
  - Driven by macroeconomic conditions and firm-specific characteristics.

- **Solution:**
  - Combine Autoencoder with a *conditional* beta model.
  - Introduce *dynamic* β using firm characteristics.



Source: Gu et al. (2021)

Table of Contents

6 Deep Learning

- No-arbitrage condition:

$$\mathbb{E}_t[M_{t+1}R^e_{i,t+1}] = 0,$$

for $t = 1, \cdots, T$ and $i = 1, \cdots, N$, where:
  - $R^e_{i,t+1}$ is the excess return of asset i at $t+1$.
  - $\mathbb{E}_t[.]$ is the expected value conditioning on the information set at t.
  - M_{t+1} is the SDF at $t+1$.

- Conditioning **down** using instruments, i.e., managed portfolios:

$$\mathbb{E}[M_{t+1}R^e_{i,t+1}g(\boldsymbol{I}_t, \boldsymbol{I}_{i,t})] = 0,$$

where $\boldsymbol{I}_t$ and $\boldsymbol{I}_{i,t}$ represents marco-economic information and firm-specific characteristics, respectively. **These managed portfolios are test assets.**

- Project the SDF on the return space:

$$M_{t+1} = 1 - \sum_{i=1}^{N} w_{i,t} R^e_{i,t+1},$$

- $w_{i,t}$ is a general function of the same information set $\boldsymbol{I}_t$ and $\boldsymbol{I}_{i,t}$:

$$w_{i,t} = w(\boldsymbol{I}_t, \boldsymbol{I}_{i,t}).$$

- In a traditional setting, we treat test assets (i.e., the choice of $g(.)$) as fixed and focus on estimating SDF weights $w_{i,t}$ to minimize pricing errors (a classic GMM problem).

- But, what if we take an *adversarial* approach to select $g(.)$ to challenge the SDF, i.e., the moment conditions the that lead to the largest pricing errors.

- The problem becomes a minimax optimization problem:

$$\min_{w} \max_{g} \frac{1}{N} \sum_{j=1}^{N} \left\| \mathbb{E}\left[\left(1 - \sum_{i=1}^{N} w(\boldsymbol{I}_t, \boldsymbol{I}_{i,t}) R^e_{i,t+1} \right) R^e_{i,t+1} g(\boldsymbol{I}_t, \boldsymbol{I}_{i,t}) \right] \right\|^2 .$$

- This problem can be modeled as a **zero-sum game**, where one player, the asset pricing modeler, aims to choose an SDF ($w(.)$ part), while the adversary searches for moment conditions ($g(.)$ part) under which the SDF fails to price.

- This can be done via a generative adversarial network (GAN). This is the work of Chen et al. (2024).

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

GAN Framework for Asset Pricing

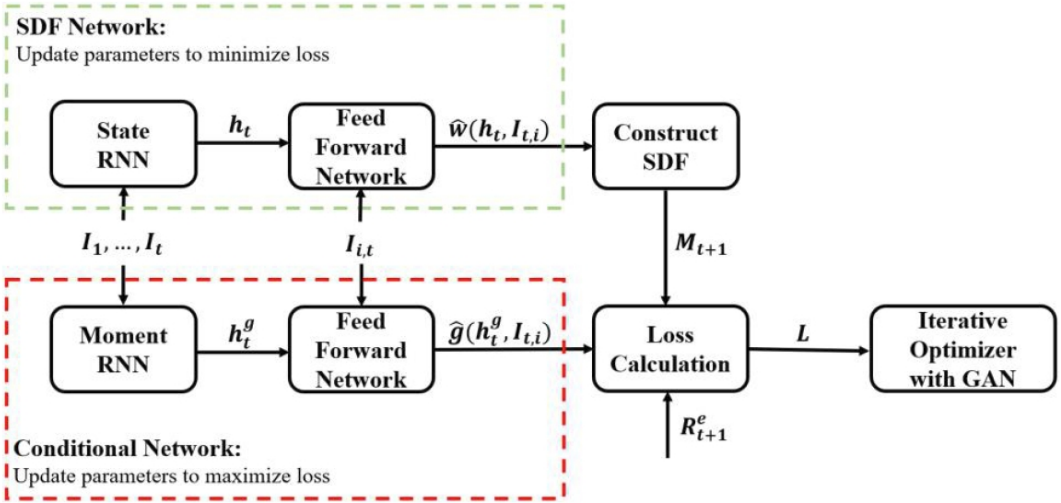
- There are two competing neural networks:
  1. **SDF Model**: Minimizes pricing errors of test assets.
  2. **Adversarial network**: Selects managed portfolios (test assets) to maximize pricing errors.

- Iterative process:
  1. w is updated by minimizing the loss function given g: $\hat{w} = \min_w L(w|g)$.
  2. g is updated by maximizing the loss given w: $\hat{g} = \max_g L(g|w)$.

- We first find portfolios that are the most mispriced and then tune the SDF to also price these assets.

- The process is repeated until the adversary cannot find portfolios with large enough pricing errors.

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.



Source: Chen et al. (2024).

GAN – Some Empirical Results

	SR			EV			Cross-Sectional R^2		
Model	Train	Valid	Test	Train	Valid	Test	Train	Valid	Test
LS	1.80	0.58	0.42	0.09	0.03	0.03	0.15	0.00	0.14
EN	1.37	1.15	0.50	0.12	0.05	0.04	0.17	0.02	0.19
FFN	0.45	0.42	0.44	0.11	0.04	0.04	0.14	-0.00	0.15
GAN	2.68	1.43	0.75	0.20	0.09	0.08	0.12	0.01	0.23

Source: Chen et al. (2024).

Table of Contents

7 Reference

References

8 Reference

Avramov, D., S. Cheng, and L. Metzker (2023). Machine learning vs. economic restrictions: Evidence from stock return predictability. *Management Science 69*(5), 2587–2619.

Baba-Yara, F., B. H. Boyer, and C. Davis (2021). The factor model failure puzzle. Working paper, Indiana University, Brigham Young University.

Belkin, M., D. Hsu, S. Ma, and S. Mandal (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *PNAS 116*(32), 15849–15854.

Chen, L., M. Pelger, and J. Zhu (2024). Deep learning in asset pricing. *Management Science 70*(2), 714–750.

Didisheim, A., S. Ke, B. T. Kelly, and S. Malamud (2023). Complexity in factor pricing models. Technical report. Working Paper.

Gu, S., B. T. Kelly, and D. Xiu (2020). Empirical asset pricing via machine learning. *Review of Financial Studies 33*(5), 2223–2273.

Gu, S., B. T. Kelly, and D. Xiu (2021). Autoencoder asset pricing models. *Journal of Econometrics 222*(1), 429–450.

Hastie, T., A. Montanari, S. Rosset, and R. J. Tibshirani (2022). Surprises in high-dimensional ridgeless least squares interpolation. *Annals of Statistics 50*(2), 949–986.

Hoerl, A. E. and R. W. Kennard (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics 12*(1), 55–67.

Jensen, T. I., B. T. Kelly, S. Malamud, and L. H. Pedersen (2022). Machine learning and the implementable efficient frontier. Technical Report 22-63, Swiss Finance Institute.

References

8 Reference

Kelly, B. T., S. Malamud, and L. H. Pedersen (2023). Principal portfolios. *Journal of Finance 78*(1), 347–387.

Kelly, B. T., S. Pruitt, and Y. Su (2019). Characteristics are covariances: A unified model of risk and return. *Journal of Financial Economics 134*(3), 501–524.

Kozak, S., S. Nagel, and S. Santosh (2018). Interpreting factor models. *Journal of Finance 73*(3), 1183–1223.

Kozak, S., S. Nagel, and S. Santosh (2020). Shrinking the cross-section. *Journal of Financial Economics 135*(2), 271–292.

Lopez de Prado, M. (2018). *Advances in Financial Machine Learning*. Hoboken, New Jersey: John Wiley & Sons, Inc.

Tibshirani, R. (1996). Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society. Series B (Methodological) 58*(1), 267–288.

Machine Learning in Factor Investing

Questions?